The Computing Atlas

Programming and Software Engineering

Volume 2

Aftab Hussain

A technical reference | Work in progress

Contents

1 Object-Oriented Design **1**
1.1 Interfaces and Protocols in Python 1
1.1.1 The design problem: an agent needs search, not a search engine 1
1.1.2 Expressing the contract with a protocol 1
1.1.3 Connecting the consumer to an implementation 2
1.1.4 How the contract is checked 3
1.1.5 From interchangeability to polymorphism 3

2 Concurrency and Asynchronous Programming **5**
2.1 Coroutines and the `asyncio` Event Loop 5
2.1.1 Coroutine functions and coroutine objects 5
2.1.2 The event loop, tasks, and `await` 5
2.1.3 Example: what runs while another task waits? 6
2.1.4 Two distinctions that prevent common mistakes 7

Chapter 1

Object-Oriented Design

1.1 Interfaces and Protocols in Python

1.1.1 The design problem: an agent needs search, not a search engine

Consider an agent that answers questions using passages retrieved from documents. To obtain these passages, it calls a search component. A FAISS-based store may serve this role during development, while a database-backed store may be used in another deployment. Tests may instead use a small implementation that returns predefined passages. Although these components perform their work differently, the agent requires the same operation from each: given a question, return relevant evidence.

In object-oriented terms, the `Agent` is a consumer of search functionality, and a class such as `FaissStore` is its provider. An object is an instance of a class; a method is a function associated with such an object. If `Agent` is written to construct or otherwise rely on the details of a particular store, switching to another provider can require changes to the agent itself. A more flexible design starts by stating only what search behavior the agent needs. This statement of expected behavior is called an *interface contract*.

1.1.2 Expressing the contract with a protocol

For this agent, the contract can be expressed as a Python method named `search()` that receives a query, a limit on the number of results, and a minimum similarity score, then returns a list of evidence objects. Python's `Protocol` provides a way to declare this requirement without implementing the search algorithm.

Listing 1.1: Declaring the search interface with a Python protocol

```
from dataclasses import dataclass
from typing import Protocol

@dataclass(frozen=True)
class Evidence:
    text: str

class SearchAdapter(Protocol):
    # Required method, not a search implementation.
    def search(
        self, query: str, top_k: int, min_score: float
    ) -> list[Evidence]: ...
```

The declaration in Listing 1.1 is a *method signature*: it specifies the method name, parameters, and expected return type. The annotation `query: str` calls for a string; `top_k: int` and `min_score: float` describe an integer result limit and a floating-point threshold; and `-> list[Evidence]` describes a list of `Evidence` objects as the result. The `self` parameter refers to the object on which the method is called and is supplied automatically when calling a bound method. The final `...` is Python's `Ellipsis` object, used here as a placeholder because the protocol specifies an operation but does not perform it.

A class satisfies this protocol when it provides a compatible `search()` method, even if the class does not inherit from `SearchAdapter`. This rule is called *structural typing*: compatibility is determined by the available methods and their types, rather than by an explicitly declared

parent class. The annotations give readers and static type-checking tools a contract to inspect; Python does not automatically enforce that contract whenever an object is passed to a function.

1.1.3 Connecting the consumer to an implementation

The next step is to give the agent an object that actually performs search. Listing 1.2 uses a simplified `FaissStore` class to make the connection visible. Its `search` method returns a fixed example rather than consulting a FAISS index, and its `Evidence` class contains only the field needed for this demonstration. A real agentic system can have additional fields and retrieval logic. The illustrative `Evidence` is a dataclass, whose annotated fields determine its generated constructor; `frozen=True` prohibits later field reassignment.

Listing 1.2: Supplying a concrete search provider to the agent

```
from dataclasses import dataclass
from typing import Protocol

@dataclass(frozen=True)
class Evidence:
    text: str

class SearchAdapter(Protocol):
    def search(self, query: str, top_k: int,
               min_score: float) -> list[Evidence]: ...

class FaissStore:
    # Compatible method: same parameters and result type.
    def search(self, query: str, top_k: int,
               min_score: float) -> list[Evidence]:
        print("FAISS search invoked")
        return [Evidence(text="Sample retrieved passage")]

class Agent:
    def __init__(self, searcher: SearchAdapter) -> None:
        self.searcher = searcher

    def ask(self, question: str) -> list[Evidence]:
        return self.searcher.search(question, 4, 0.18)

store = FaissStore()
agent = Agent(store)
results = agent.ask("How do tokens work?")
print(results[0].text)
```

The agent initializer receives the provider and saves it as `self.searcher`. This initializer is named `Agent.__init__()`. The parameter annotation `searcher: SearchAdapter` expresses what the agent needs; it does not create a `SearchAdapter` object. The caller instead creates a `FaissStore` instance and passes it to `Agent(store)`. Supplying a required component from outside its consumer is called *dependency injection*. It lets the caller choose the provider without changing the agent's implementation.

When `agent.ask()` executes, `self.searcher` refers to the `FaissStore` instance. Python therefore invokes `FaissStore.search()` directly. The protocol is not an intermediary through which requests pass; it specifies the expected interface.

1.1.4 How the contract is checked

The difference between a declared interface and an executed method is important. A static type checker, such as Pyright or mypy, can compare the methods of a proposed provider with the `SearchAdapter` contract before the program runs. Python itself uses the concrete object at runtime. The following example makes the distinction concrete.

To see where this distinction matters, suppose a provider has the required method name but accepts too few arguments:

Listing 1.3: A method signature that does not satisfy the contract

```
class BadStore:
    # Omits the required top_k and min_score arguments.
    def search(self, query: str) -> list[Evidence]:
        return []

agent = Agent(BadStore())     # Static checker: incompatible
agent.ask("Authentication?") # Runtime: TypeError
```

A checker flags `Agent(BadStore())` because the supplied object's `search()` method cannot accept the three arguments required by the interface. If the code runs without static checking, construction of the agent still succeeds: the annotation is not a runtime guard. The failure occurs when `Agent.ask()` calls the method with three explicit arguments, raising a `TypeError`; Python also passes `self` implicitly. A method with the correct parameters but the wrong return type, such as `str` instead of `list[Evidence]`, can likewise be reported by a type checker. By contrast, even a correctly typed method may retrieve irrelevant passages. Type checking verifies interface compatibility, not the correctness of the search results.

Figure 1.1 summarizes why the type-level contract and the runtime call must be distinguished.

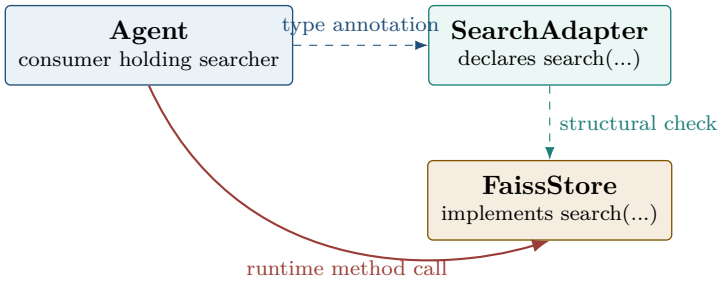


Figure 1.1: The dashed arrows represent type-level relationships. The solid arrow represents the actual call from the agent to the supplied provider; no call passes through the protocol.

1.1.5 From interchangeability to polymorphism

The value of separating the agent from a concrete store becomes clearer when another provider is introduced. For example, a PostgreSQL-backed store can implement the same `search()` signature, and the existing agent can receive it without changing `Agent.ask()`:

Listing 1.4: Changing the provider without changing the consumer

```
class PostgreSQLStore:
    def search(self, query: str, top_k: int,
               min_score: float) -> list[Evidence]:
        return [Evidence(text="Sample database passage")]
```

```
5
6  agent = Agent(PostgreSQLStore())
7  results = agent.ask("How do tokens work?")
```

The same `Agent.ask()` operation now invokes `PostgreSQLStore.search()` rather than `FaissStore.search()`. This ability to use different concrete objects through a shared interface is *polymorphism*. Python selects the method belonging to the object held by `self.searcher`, so the consumer's call remains unchanged while the behavior of the supplied provider can differ.

This design also illustrates the *dependency inversion principle*. The agent is the high-level component, concerned with obtaining evidence for question answering; the storage engine is a lower-level implementation detail. Instead of making the agent depend on FAISS-specific operations, both sides conform to the search contract the agent needs. Thus, the high-level logic remains independent of a particular retrieval technology. Dependency injection is the mechanism used here to supply the chosen object; polymorphism describes how interchangeable objects respond to the same method call; and dependency inversion describes the resulting direction of design dependencies. These concepts are related, but not synonymous.

Chapter 2

Concurrency and Asynchronous Programming

2.1 Coroutines and the `asyncio` Event Loop

Programs often wait for operations to finish, such as a network response, a file operation, or a timer. Python's `asyncio` library lets a program make progress on another task while one task is waiting. The key ideas are coroutines, tasks, an event loop, and `await`.

2.1.1 Coroutine functions and coroutine objects

A coroutine is a computation that can pause and resume. A function written with `async def` is called a *coroutine function*. Calling that function creates a *coroutine object*; unlike calling a function written with `def`, this call does not immediately execute the function body. The coroutine body executes when an event loop advances it, for example by awaiting it or scheduling it as a task.

Listing 2.1: Creating a coroutine object versus executing it

```
1 import asyncio
2
3 async def greet():                # Coroutine function
4     print("Hello")               # Body has not run yet
5
6 operation = greet()               # Coroutine object; prints nothing
7 asyncio.run(operation)            # Event loop runs it; prints Hello
```

In Listing 2.1, `greet()` creates a coroutine object, which represents work to be executed; it does not execute the print statement. `asyncio.run(operation)` starts an event loop, executes the coroutine until it finishes, and closes the loop. Simply creating a coroutine object without awaiting or scheduling it does not run the operation.

2.1.2 The event loop, tasks, and `await`

The *event loop* is the runtime scheduler used by `asyncio`. It runs tasks that are ready to execute, keeps track of operations that are waiting (for example, timers or I/O), and resumes waiting tasks when they become ready. It does not require a separate operating-system thread for each task.

A `Task` is an `asyncio.Task` object that wraps a coroutine object. This is a wrapper object, not another function. Here, "wraps" means the `Task` holds and manages the coroutine: it arranges for the event loop to execute it and tracks its completion, result, exception, and cancellation state. For example, `asyncio.create_task(greet())` creates a `Task` from the coroutine object returned by `greet()` and schedules it to run. A `Task` is not the same thing as a new thread.

Inside a coroutine, `await` waits for an awaitable, such as another coroutine or a `Task`. If that operation is not yet ready, `await` suspends the current coroutine and gives the event loop a chance to run other ready tasks. The statement after `await` runs only when the awaited operation completes. If the result is already available, `await` may continue without suspending.

2.1.3 Example: what runs while another task waits?

Listing 2.2 makes the scheduling explicit: `worker_a()` and `worker_b()` first produce coroutine objects, `create_task()` wraps and schedules them, and `gather()` lets `main()` wait for both results.

Listing 2.2: Two coroutines progressing concurrently on one event loop

```
import asyncio

async def worker_a():
    print("A: Starting")
    await asyncio.sleep(3)       # A waits; the event loop is free
    print("A: Finished")         # Runs after about 3 seconds

async def worker_b():
    print("B: Starting")
    await asyncio.sleep(1)       # B yields control while waiting
    print("B: Finished")         # Runs after about 1 second

async def main():
    coro_a = worker_a()          # Create coroutine; A has not run
    coro_b = worker_b()          # Create coroutine; B has not run
    task_a = asyncio.create_task(coro_a) # Wrap and schedule A
    task_b = asyncio.create_task(coro_b) # Wrap and schedule B
    await asyncio.gather(task_a, task_b) # Main waits for both

asyncio.run(main())              # Create, run, and close event loop
```

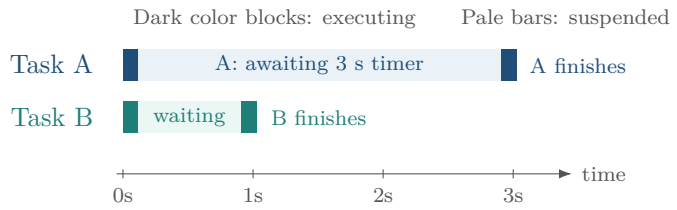


Figure 2.1: Approximate timeline for Listing 2.2. Both tasks start near $t = 0$. While A waits for its three-second timer, B can run and complete after approximately one second.

Execution sequence. When `asyncio.run(main())` starts the event loop, `main()` creates two coroutine objects and schedules two `Tasks`. `main()` then reaches `await asyncio.gather(...)` and waits for both `Tasks`. The event loop runs A until `await asyncio.sleep(3)` suspends it, then runs B until `await asyncio.sleep(1)` suspends it. After about one second, B's timer finishes and B prints its final line. After about three seconds, A prints its final line (Figure 2.1). The output is:

```
A: Starting
B: Starting
B: Finished
A: Finished
```

The total elapsed time is about three seconds, rather than four, because the two waits overlap. This is *concurrency*: the tasks make progress during overlapping intervals. It does not imply that two Python statements execute simultaneously on different CPU cores.

2.1.4 Two distinctions that prevent common mistakes

Awaiting is not the same as scheduling concurrently. If `main()` instead used the following code, A would finish before B starts; the elapsed time would be about four seconds:

```
async def main():
    await worker_a()            # Wait for all of A first
    await worker_b()            # Only then start B
```

Removing `await` does not make an asynchronous operation run in the background. For instance, writing `asyncio.sleep(3)` without `await` merely creates a coroutine object for the sleep; it does not start a timer that independently pauses the function. Python may warn that the coroutine was never awaited. Conversely, `await` does not automatically create a new `Task`: in the concurrent example, `create_task()` performs the scheduling, and `await` lets `main()` wait while those `Task`s execute.

Takeaway. `async def` defines a coroutine function. Calling it creates a coroutine object. `create_task()` schedules that object within a `Task` wrapper, and the event loop runs ready `Task`s. At an `await` that must wait, a coroutine suspends so other ready `Task`s can progress.